與Trident營運者一起部署

Astra Trident

NetApp
May 11, 2022

This PDF was generated from https://docs.netapp.com/zh-tw/trident-2110/trident-get-started/kubernetes-customize-deploy.html on May 11, 2022. Always check docs.netapp.com for the latest.

目錄

與Trident營運者一起部署 1
使用Helm部署Trident運算子 1
手動部署Trident運算子 2
自訂Trident營運者部署 6

與Trident營運者一起部署

您可以與Trident運算子一起部署Astra Trident。您可以手動或使用Helm來部署Trident運算子。

> 如果您尚未熟悉 "基本概念"現在正是這麼做的好時機。

若要部署Astra Trident、必須符合下列先決條件：

- 您擁有執行Kubernetes 1.17及更新版本之受支援Kubernetes叢集的完整權限。
- 您可以存取支援的NetApp儲存系統。
- 您可以從所有Kubernetes工作節點掛載磁碟區。
- 您的Linux主機已安裝並設定為管理您要使用的Kubernetes叢集。
- 您已將「KUBECCONFIG」環境變數設定為指向Kubernetes叢集組態。
- 您已啟用 "具備Astra Trident所需的閘道"。
- 如果您使用Kubernetes搭配Docker Enterprise、 "請依照他們的步驟啟用CLI存取"。

您知道嗎？太棒了！讓我們開始吧。

使用Helm部署Trident運算子

使用Helm執行列出的步驟來部署Trident運算子。

除了上述先決條件之外、若要使用Helm部署Trident運算子、您還需要下列項目：

- Kubernetes 1.17及更新版本
- Helm版本3

步驟

1. 請從下載安裝程式套裝組合 "Trident GitHub" 頁面。安裝程式套裝組合在「/helm」目錄中包含Helm圖表。
2. 使用「helm install」命令並指定部署名稱。請參閱下列範例：

   ```
   helm install <name> trident-operator-21.07.1.tgz --namespace <namespace
   you want to use for Trident>
   ```

   如果您尚未建立Trident的命名空間、可以將「-cree-namespace"參數新增至「helm install」命令。然後、Helm會自動為您建立命名空間。

   安裝期間有兩種傳遞組態資料的方法：

   - `--Values`（或-f）：指定具有置換的Yaml檔案。這可以多次指定、最右邊的檔案會優先。
   - -Set'：在命令列上指定置換項目。

例如、若要變更預設值「debug」、請執行下列「-set」命令：

```
$ helm install <name> trident-operator-21.07.1.tgz --set tridentDebug=true
```

Helm圖表中的「values.yaml」檔案提供金鑰清單及其預設值。

「helm清單」會顯示安裝的詳細資料、例如名稱、命名空間、圖表、狀態、 應用程式版本、修訂編號等。

手動部署Trident運算子

執行所列步驟、手動部署Trident運算子。

步驟1：判斷Kubernetes叢集的資格

首先您需要登入Linux主機、然後確認它正在管理_運作_、 "支援的Kubernetes叢集" 您擁有必要的權限。

> 使用OpenShift時、請在所有範例中使用「oc"而非「kubecll」、然後先執行「ocLogin -u system:admin」或「occ login-u kube-admin」、以*系統：admin*登入。

若要查看Kubernetes版本是否晚於1.17、請執行下列命令：

```
kubectl version
```

若要查看您是否具有Kubernetes叢集管理員權限、請執行下列命令：

```
kubectl auth can-i '*' '*' --all-namespaces
```

若要驗證是否可以從Docker Hub啟動使用映像的Pod、並透過Pod網路連線至儲存系統、請執行下列命令：

```
kubectl run -i --tty ping --image=busybox --restart=Never --rm -- \
  ping <management IP>
```

步驟2：下載並設定營運者

> 從21.01開始、Trident運算子就是叢集範圍。若要使用Trident營運者來安裝Trident、必須建立「TridentOrchestrator」自訂資源定義（CRD）並定義其他資源。在安裝Astra Trident之前、您應該先執行這些步驟來設定操作員。

1. 下載最新版本的 "Trident安裝程式套裝組合" 從_下載_區段擷取。

```
wget https://github.com/NetApp/trident/releases/download/v21.04/trident-installer-21.04.tar.gz
tar -xf trident-installer-21.04.tar.gz
cd trident-installer
```

2. 使用適當的CRD資訊清單來建立「TridentOrchestrator」CRD。之後您將建立一個「TridentOrchestrator」自訂資源、以產生操作者的安裝。

   執行下列命令：

```
kubectl create -f
deploy/crds/trident.netapp.io_tridentorchestrators_crd_post1.16.yaml
```

3. 建立「TridentOrchestrator」CRD之後、請建立下列作業者部署所需的資源：

   - 營運者的服務帳戶
   - 叢集角色和叢集角色繫結至服務帳戶
   - 專屬的PodSecurity原則
   - 營運者本身

     Trident安裝程式包含定義這些資源的資訊清單。根據預設、運算子會部署在「Trident」命名空間中。如果「Trident」命名空間不存在、請使用下列資訊清單來建立名稱空間。

```
$ kubectl apply -f deploy/namespace.yaml
```

4. 若要在預設的「三叉」命名空間以外的命名空間中部署運算子、您應該更新「serviceaccount.yaml」、「clusterrolebind.yaml」和「oper.yaml」等資訊清單、然後產生「bundle.yaml」。

   執行下列命令、以更新Yaml清單、並使用「kustomization、yaml」產生您的「bunder.yaml」：

```
kubectl kustomize deploy/ > deploy/bundle.yaml
```

   執行下列命令以建立資源並部署營運者：

```
kubectl create -f deploy/bundle.yaml
```

5. 若要在部署後驗證操作員的狀態、請執行下列步驟：

```
$ kubectl get deployment -n <operator-namespace>
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
trident-operator    1/1     1            1           3m

$ kubectl get pods -n <operator-namespace>
NAME                                READY   STATUS             RESTARTS
AGE
trident-operator-54cb664d-lnjxh     1/1     Running            0
3m
```

營運者部署成功建立一個在叢集中其中一個工作節點上執行的Pod。

> Kubernetes叢集中只應有*一個運算子執行個體*。請勿建立Trident營運者的多個部署。

步驟3：建立TridentOrchestrator並安裝Trident

您現在可以使用運算子來安裝Astra Trident！這需要建立「TridentOrchestrator」。Trident安裝程式隨附建立「TridentOrchestrator」的範例定義。這將開始在「Trident」命名空間中安裝。

```
$ kubectl create -f deploy/crds/tridentorchestrator_cr.yaml
tridentorchestrator.trident.netapp.io/trident created

$ kubectl describe torc trident
Name:        trident
Namespace:
Labels:      <none>
Annotations: <none>
API Version: trident.netapp.io/v1
Kind:        TridentOrchestrator
...
Spec:
  Debug:     true
  Namespace: trident
Status:
  Current Installation Params:
    IPv6:                      false
    Autosupport Hostname:
    Autosupport Image:         netapp/trident-autosupport:21.04
    Autosupport Proxy:
    Autosupport Serial Number:
    Debug:                     true
    Enable Node Prep:          false
    Image Pull Secrets:
    Image Registry:
    k8sTimeout:           30
    Kubelet Dir:          /var/lib/kubelet
    Log Format:           text
    Silence Autosupport:  false
    Trident Image:        netapp/trident:21.04.0
  Message:                  Trident installed  Namespace:
trident
  Status:                   Installed
  Version:                  v21.04.0
Events:
    Type Reason Age From Message ---- ------ ---- ---- -------Normal
    Installing 74s trident-operator.netapp.io Installing Trident Normal
    Installed 67s trident-operator.netapp.io Trident installed
```

Trident運算子可讓您使用「TridentOrchestrator」規格中的屬性、自訂Astra Trident的安裝方式。請參閱 "自訂您的Trident部署"。

「TridentOrchestrator」的狀態會指出安裝是否成功、並顯示安裝的Trident版本。

狀態	說明
安裝	營運者使用此「TridentOrchestrator」CR來安裝Astra Trident。
已安裝	Astra Trident已成功安裝。
正在解除安裝	操作員正在解除安裝Astra Trident、因為是「shpec.uninstall=true」。
已解除安裝	Astra Trident已解除安裝。
失敗	營運者無法安裝、修補、更新或解除安裝Astra Trident；營運者將自動嘗試從此狀態恢復。如果此狀態持續存在、您將需要疑難排解。
正在更新	營運者正在更新現有的安裝。
錯誤	不使用「TridentOrchestrator」。另一個已經存在。

在安裝過程中、「TridentOrchestrator」的狀態會從「安裝」變更為「安裝」。如果您觀察到「失敗」狀態、而操作者無法自行恢復、則應檢查操作者的記錄。請參閱 "疑難排解" 區段。

您可以查看已建立的Pod、確認Astra Trident安裝是否已完成：

```
$ kubectl get pod -n trident
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-7d466bf5c7-v4cpw        5/5     Running   0           1m
trident-csi-mr6zc                   2/2     Running   0           1m
trident-csi-xrp7w                   2/2     Running   0           1m
trident-csi-zh2jt                   2/2     Running   0           1m
trident-operator-766f7b8658-ldzsv   1/1     Running   0           3m
```

您也可以使用「tridentctl」來檢查安裝的Astra Trident版本。

```
$ ./tridentctl -n trident version
+----------------+----------------+
| SERVER VERSION | CLIENT VERSION |
+----------------+----------------+
| 21.04.0        | 21.04.0        |
+----------------+----------------+
```

現在您可以繼續建立後端。請參閱 "部署後工作"。

如需部署期間的疑難排解問題、請參閱 "疑難排解" 區段。

自訂Trident營運者部署

Trident運算子可讓您使用「TridentOrchestrator」規格中的屬性、自訂Astra Trident的安裝方式。

請參閱下表以取得屬性清單：

參數	說明	預設
命名空間	用於安裝Astra Trident的命名空間	"預設"
「Debug」	啟用Astra Trident的偵錯功能	錯
《IPv6》	透過IPv6安裝Astra Trident	錯
「k8stimeout」	Kubernetes作業逾時	30秒
「ilenceAutoSupport」	請勿AutoSupport 自動將此套裝組合傳送至NetApp	錯
「enableNodePrep」	自動管理工作節點相依性（* BETA *）	錯
「autosupportImage」	遙測的容器影像AutoSupport	「NetApp/Trident自動支援：21.04.0」
「autosupportProxy」	代理伺服器的位址/連接埠、用於傳送AutoSupport 「遙測」功能	"<a href="http://proxy.example.com:8888"" class="bare">http://proxy.example.com:8888"</a>
解除安裝	用來解除安裝Astra Trident的旗標	錯
記錄格式	要使用的Astra Trident記錄格式[text、json]	"文字"
「TridentImage」	要安裝的Astra Trident映像	「NetApp/Trident：21.04」
「影像登錄」	內部登錄的路徑、格式為「<登錄FQDN >[:port]（/子路徑）」	"k8s.gcr.io/sig-storage（k8s 1.17+）或kay.io/k8scsi"
《kubeletDir	主機上的kubelet目錄路徑	"/var/lib/kubelet"
《有線》	要刪除以執行完整移除Astra Trident的資源清單	
「imagePullSecrets」	從內部登錄擷取映像的機密	

「pec.namespace`」在「TridentOrchestrator」中指定、以表示要安裝哪個命名空間Astra Trident。此參數*無法在安裝Astra Trident之後更新*。嘗試這樣做會導致「TridentOrchestrator」狀態變更為「失敗」。Astra Trident不打算跨命名空間移轉。

自動工作節點準備是*測試版功能*、僅適用於非正式作業環境。

您可以在定義「TridentOrchestrator」時、使用上述屬性來自訂安裝。範例如下：

```
$ cat deploy/crds/tridentorchestrator_cr_imagepullsecrets.yaml
apiVersion: trident.netapp.io/v1
kind: TridentOrchestrator
metadata:
  name: trident
spec:
  debug: true
  namespace: trident
  tridentImage: netapp/trident:21.04.0
  imagePullSecrets:
  - thisisasecret
```

如果您想要自訂超出「TridentOrchestrator」引數範圍的安裝、您應該考慮使用「tridentctl」來產生可視需要修改的自訂Yaml資訊清單。

Copyright Information

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

Trademark Information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.